

VERIQO PILOT MODE AND DEPLOYMENT READINESS

This document answers Commercialization Sprint items 10 and 18-20 by name: Pilot Mode (item 18), minimal multi-tenancy (item 19), observability (item 20), and the production security blocker classification (item 10). It reports the real, current state of the codebase honestly -- every "DONE" line below names the exact code and test that backs it; every "GAP" line names the exact thing that does not exist yet, rather than being silently assumed.

ITEM 18 -- PILOT MODE CHECKLIST

Capability	Status	Real backing
------------	--------	--------------

--- --- ---

Tenant isolation	DONE	pkg/commercial/api.Store keys every case/evidence record by tenantID+"/"+ID (caseKey/evidenceKey) -- a cross-tenant lookup fails at "not found" before any ownership check runs. See item 19 below for the full detail and its adversarial tests.
------------------	------	---

Case isolation	DONE	Same mechanism as tenant isolation -- each case's evidence, decision, action, and dossier are scoped to that one tenantID+"/"+caseID key; nothing about one case leaks into another's CaseView, Dossier, or Replay result.
----------------	------	--

Audit	DONE	pkg/platform/audit.AuditStore (hash-chained, Auditor{}.VerifyChain-checkable) backs both the Commercial API's own Decision/Action ledger writes (Store.Ledger()) and every HTTP request via security.AuditMiddleware (veriqo/gateway/rest/server.go).
-------	------	---

RBAC	DONE	security.RBACMiddleware (pkg/platform/security/security.go) gates route prefixes by JWT role; proven for the insurance routes in insurance_decide_route_test.go (TestInsuranceDecideRouteRejectsWrongRole). The Commercial API v1 routes compose the SAME middleware stack in NewServer -- they are not a second, parallel auth system.
------	------	---

API keys	DONE	security.APIKeyMiddleware, composed in NewServer alongside JWT/RBAC.
----------	------	--

OIDC	GAP	No OpenID Connect integration exists. security.JWTMiddleware verifies a locally HS256-signed token (security.SignHS256), not a third-party IdP's token. A pilot customer wanting SSO via their own IdP needs this built; it is not a redesign of the existing JWT layer, just a second, additional verification path.
------	-----	---

Evidence retention	PARTIAL / GAP	pkg/governance/data implements a real retention/purge lifecycle (ACTIVE -> RETENTION_ELIGIBLE -> HELD -> REDACTION_REQUIRED -> REDACTED -> PURGE_ELIGIBLE -> PURGED) that preserves audit integrity even after content is purged, and pkg/insurance/preservation implements legal-hold orders. Neither is wired into pkg/commercial/api.Store's evidence path yet -- a Commercial API v1 caller today has no way to place a hold or trigger retention on evidence they submitted. The mechanism exists; the integration does not.
--------------------	---------------	---

Export	DONE	dossier.WriteMachinePackage (the Evidence Dossier v1 "Machine" form) and Store.WriteDossierPackage/GET /v1/cases/{id}/dossier?format=package -- a real .zip a customer can download and independently verify.
--------	------	---

Usage metrics	DONE	pkg/commercial/telemetry + GET /v1/metrics -- see item 20 below.
---------------	------	--

Health	PARTIAL	GET /healthz reports {"status":"ok","persistence":...}. It is a single combined check, not separate liveness/readiness probes -- see item 20.
--------	---------	---

Logging	DONE	loggingMiddleware (veriqo/gateway/rest/server.go) logs method/path/status/latency for every request, with explicit log-injection sanitization (strconv.Quote on the request path).
---------	------	--

Support diagnostics	GAP	No dedicated support/diagnostics endpoint (e.g. a bundle of recent logs + config + version for a support ticket) exists. GET /healthz and GET /v1/metrics are the closest real artifacts a support engineer could use today.
---------------------	-----	--

ITEM 19 -- MINIMAL MULTI-TENANCY

What the reviewer's model names: Tenant -> Users / Cases / Evidence / Policies / Decisions / Actions / Dossiers, with the Tenant ID cryptographically/authorization-bound on every path.

What actually exists:

- Cases, Evidence, Decisions, Actions, Dossiers are all real, tenant-scoped entities in pkg/commercial/api.Store -- every one of CreateCase, SubmitEvidence, DecideCase, ActOnCase, GenerateDossier, Replay takes a tenantID and enforces it via the key-namespacing mechanism described above.
- Users and Policies are NOT first-class tenant-scoped entities in this codebase. security.RBACMiddleware's RoleTable is global (one role-to-path-prefix map for the whole deployment), and JWT Claims.Subject/Claims.Role identify a caller but are not themselves tied to a specific tenant record anywhere in Store. This is the honest gap named in commercial_v1_routes.go's own doc comment: `"TenantID is currently a caller-supplied field (request body or query parameter), not yet cryptographically bound to the verified JWT identity."` A caller who knows another tenant's ID can currently supply it in a request and reach Store's tenant-scoped isolation logic on THAT tenant's behalf -- isolation between tenants once inside Store is real and tested, but nothing yet stops a caller from claiming to be a different tenant than their JWT identity actually maps to. Closing this requires a real tenant-identity registry (Users/Tenant mapping) and binding TenantID to the verified JWT subject at the middleware layer -- named here as the next integration step, not silently assumed done.

The three mandatory tests the reviewer names, and where they live:

Requirement	Test
--- ---	
Tenant A cannot read Tenant B	TestTenantAISolationFromTenantB
(pkg/commercial/api/store_test.go, 8 sub-tests) and	
TestCommercialV1RoutesTenantIsolationOverHTTP	
(veriqo/gateway/rest/commercial_v1_routes_test.go) -- both prove a 404, not tenant B's data, when tenant A requests tenant B's case or evidence by ID.	
Tenant A cannot modify Tenant B	Same TestTenantAISolationFromTenantB sub-tests
cannot_modify_via_decide and cannot_act_on_case;	
TestCommercialV1RoutesTenantIsolationOverHTTP proves the same over HTTP for POST /v1/cases/{id}/decide.	
Tenant A cannot replay Tenant B authorization	TestTenantAISolationFromTenantB's
cannot_replay_authorization sub-test.	

ITEM 20 -- OBSERVABILITY

Signal	Status	Real backing
--- --- ---		
Metrics	DONE	pkg/commercial/telemetry.Metrics, exposed at GET /v1/metrics. Every one of the 9 named counters is wired to a real Store branch (see that package's own doc comment for exactly which): evidence_ingestion_total, evidence_verification_failures, custody_chain_failures, decision_latency (as decision_latency_avg_millis + decision_count), authorization_denials, action_failures, ledger_commit_failures, replay_failures, external_adapter_failures. TestMetricsReflectRealActivity (store_test.go) and TestCommercialV1RoutesMetricsReflectRealActivity (commercial_v1_routes_test.go) prove these move with real activity, not statically. Honest exception: external_adapter_failures always reads zero in this reference build -- item 11's real insurer/adjuster/P&I/AIS integrations

are external pilot integrations, not built in this engagement, so nothing calls an external adapter yet for this counter to ever increment on.

Logs DONE loggingMiddleware, per-request, sanitized against log injection.

Traces GAP No distributed tracing (OpenTelemetry spans or equivalent) exists. A request's path through JWT -> RBAC -> APIKey -> handler -> Store is visible only as one log line today, not as a span tree.

Health PARTIAL One combined GET /healthz, not separate /readyz (ready to accept traffic) and /livez (process is alive, restart if not) endpoints a real orchestrator (Kubernetes, etc.) would want to probe independently.

Readiness / Liveness GAP See above -- no separate liveness/readiness distinction exists yet.

Audit DONE Covered under item 18 above; the same pkg/platform/audit.AuditStore backs both business-decision audit and HTTP-request audit.

Alerts GAP No alerting integration (paging, threshold-based notification) exists. GET /v1/metrics' real counters are what a pilot deployment's own monitoring stack would need to scrape and alert on -- this codebase does not push alerts itself.

ITEM 10 -- PRODUCTION SECURITY BLOCKER CLASSIFICATION

This engagement's earlier rounds already built and honestly tracked the 8 production blockers item 10 asks for -- see docs/governance/production-blockers.json (the machine-readable contract) and evidence/blockers-qualification-report.json (the live, independently-recomputed qualification result, not a stale label). Reproduced here for this document's own completeness, in item 10's own MUST-CLOSE vs. CAN-BE-PILOT-ENVIRONMENT framing:

MUST CLOSE BEFORE PAID PILOT (per item 10's own list):

- HSM/KMS-backed production signing -- blocker hsm_kms
- Durable persistent ledger -- no dedicated blocker entry exists for this specific item; honestly, pkg/commercial/api.Store is explicitly documented as an in-memory reference implementation (see that package's own doc comment) that loses every case and evidence record on process restart. This is a real, named gap this document is not aware of any existing blocker covering.
- Production identity/mTLS -- blocker spire_mtls
- Secure object storage + retention -- partially covered by pkg/governance/data's retention lifecycle (see item 18 above); no dedicated "secure object storage" blocker entry exists for the evidence bytes themselves (this codebase stores evidence metadata and hashes in manifest.Manifest, never the raw bytes -- object storage for the raw bytes is external to this codebase by design, and its security is a deployment-time concern, not something this repository can qualify on its own).
- Vulnerability scanning -- blocker supply_chain_scan
- Backup/restore -- no dedicated blocker entry; ties back to the durable-persistent-ledger gap above, since there is currently nothing durable in the Commercial API layer to back up.
- Production logging/monitoring -- item 20 above is the current state; genuinely partial (metrics/logs/audit real, traces/alerts/separate health probes are gaps).
- Security incident response procedure -- no dedicated blocker entry; this is an organizational/process artifact, not code, and is named here as outstanding rather than silently assumed to exist.

CAN BE PILOT ENVIRONMENT (per item 10's own list; must show NOT_YET_PRODUCTION_QUALIFIED, never hidden):

- Multi-region -- blocker multi_region_dr
- 100-node physical qualification -- blocker scale_qualification
- 72-hour soak at production scale -- blocker soak_72h
- Independent penetration test -- blocker pentest

Current status of all 8 tracked blockers (from evidence/blockers-qualification-report.json, this session's own fresh read, not a cached or stale claim): every one of pentest, scale_qualification, multi_region_dr, hsm_kms, live_data, soak_72h, spire_mtls, supply_chain_scan reads READY_FOR_REAL_QUALIFICATION. That status means: the MECHANISM this codebase would need for a real qualification run (fixture-mode and synthetic-mode test harnesses, failure-injection matrices, measurement capture) is built and proven; the REAL external qualification itself (an actual signed pentest vendor report, actual 100 physical/cloud nodes, actual procured HSM/KMS tenancy, actual multi-region infrastructure, an actual 72-hour run at production scale) has not happened, because it requires real external infrastructure and vendors this engagement does not have. Per item 10's own instruction, this is reported here as NOT_YET_PRODUCTION_QUALIFIED, not hidden or implied otherwise.

HONEST SUMMARY

Pilot Mode's tenant/case isolation, audit, RBAC, API-key auth, evidence export, and usage metrics are real and tested. OIDC, evidence-retention integration into the Commercial API path, distributed tracing, separate liveness/readiness probes, alerting, and support diagnostics are named gaps, not silently-assumed features. Every one of the 8 tracked production security blockers has a proven mechanism but no completed real-world external qualification -- this deployment is pilot-ready under the caveats above, not yet fully production-qualified, which is exactly item 10's own required framing.